

Esportazione dei dati in un file .csv

Anche se normalmente quello che si fa è importare i dati in **R** [1], talora potrebbe essere necessario esportarli da **R** sotto forma di file. La soluzione migliore è quella di salvare i dati in un file `.csv` che potrà poi essere facilmente importato in qualsiasi software esterno.

La tecnica viene illustrata impiegando la funzione **write.table()** nella quinta e ultima riga di codice di questo script. Se non esiste già, la cartella `C:\Rdati\` va creata prima di eseguire lo script.

Copiate e incollate lo script nella `Console di R` e premete ↵ Invio:

```
# ESPORTA I DATI DI UNA MATRICE
#
x <- c(4.2, 6.8, 2.5, 8.3, 5.4, 7.9, 5.3, 6.7, 2.2, 3.1) # genera l'array x
y <- c(3.1, 2.2, 6.7, 5.3, 7.9, 5.4, 8.3, 2.5, 6.8, 4.2) # genera l'array y
mymatrix <- data.frame(x, y) # combina gli array in una matrice
#
mymatrix # mostra la matrice nella quale R impiega il punto (.) come separatore dei decimali
#
write.table(mymatrix,file="C:/Rdati/esporta_csv.csv",sep=";", dec=".", quote=FALSE,
row.names=FALSE) # esporta i dati in un file di testo .csv sostituendo il punto (.) con la virgola (,)
#
```

Le prime tre righe servono semplicemente a generare la matrice [2] impiegata come esempio.

Quindi viene mostrata nella `Console di R` la matrice **mymatrix** con il punto (.) come separatore dei decimali e con gli identificativi univoci/nomi di riga (1, 2, eccetera) generati automaticamente da **R**:

```
> mymatrix # mostra la matrice nella quale R impiega il punto (.) come separatore dei decimali
      x    y
1  4.2  3.1
2  6.8  2.2
3  2.5  6.7
4  8.3  5.3
5  5.4  7.9
6  7.9  5.4
7  5.3  8.3
8  6.7  2.5
9  2.2  6.8
10 3.1  4.2
```

La funzione **write.table()** con i suoi argomenti:

- esporta i dati dell'oggetto **mymatrix**;
- genera in uscita il file **esporta_csv.csv** nella posizione **C:/Rdati/**;
- impiega come separatore di campo il punto e virgola (**sep=";"**)
- impiega come separatore dei decimali la virgola (**dec=","**)
- non chiude tra le virgolette "" i campi non numerici (**quote=FALSE**)
- non esporta i nomi delle righe (**row.names=FALSE**)

Questo è il contenuto del file `C:\Rdati\esporta_csv.csv` generato con la funzione **write.table()** e aperto con un editor di testo:

```
x;y
```

4,2;3,1
6,8;2,2
2,5;6,7
8,3;5,3
5,4;7,9
7,9;5,4
5,3;8,3
6,7;2,5
2,2;6,8
3,1;4,2

Compaiono come previsto nello script il punto e virgola (;) come separatore di campo e la virgola (,) come separatore dei decimali. Notare che grazie all'argomento **row.names=FALSE** nel file esportato non compaiono gli identificativi/nomi di riga di **R**:

Potete importare senza difficoltà il file `esporta_csv.csv` in *Excel* o in un programma appartenente alla categoria del software libero [3] come *OpenOffice calc* o *LibreOffice calc*.

Ma in realtà **R** oltre alla funzione **write.table()** fornisce altre due funzioni per esportare i dati in formato `.csv`:

→ la funzione **write.csv()** che consente di esportare i dati impiegando la virgola (,) come separatore di campo [4] e il punto (.) come separatore dei decimali e quindi impiega i separatori secondo le convenzioni vigenti nel mondo anglosassone;

→ la funzione **write.csv2()** che consente di esportare i dati impiegando il punto e virgola (;) come separatore di campo e la virgola (,) come separatore dei decimali e quindi impiega i separatori secondo le convenzioni vigenti in Italia, Spagna, Francia e in altri Paesi europei.

Per illustrare queste modalità nello script che segue viene costruita una tabella che viene esportata nella cartella `C:\Rdati\` in quattro file differenti impiegando le tre funzioni:

→ **write.table()**
→ **write.csv()**
→ **write.csv2()**

Copiate e incollate lo script nella `Console` di **R** e premete ↵ Invio:

```
# ESPORTA I DATI DI UNA TABELLA
#
# genera cinque array
id <- c("MT", "GF", "MC", "SB", "FE", "AB", "RF")
sesso <- c("M", "F", "F", "M", "F", "M", "F")
anni <- c(69, 56, 53, 28, 61, 46, 31)
peso_kg <- c(76, 63, 71, 73, 54, 92, 81)
altezza_m <- c(1.78, NA, 1.60, 1.78, 1.54, 1.84, 1.56)
mytable <- data.frame(id, sesso, anni, peso_kg, altezza_m) # combina gli array in una tabella
#
mytable # mostra la tabella nella quale R impiega il punto (.) come separatore dei decimali
#
# (a) write.table consente sia (.) sia (,) come separatore dei decimali
write.table(mytable, file="C:/Rdati/write_table_p.csv", quote=FALSE, sep=";", dec=".",
na="", col.names=TRUE, row.names=FALSE)
#
# (b) write.table consente sia (.) sia (,) come separatore dei decimali
write.table(mytable, file="C:/Rdati/write_table_v.csv", quote=FALSE, sep=";", dec=".",
na="", col.names=TRUE, row.names=FALSE)
#
# (c) write.csv usa (,) come separatore di campo e (.) come separatore dei decimali
```

```

write.csv(mytable, file="C:/Rdati/write_csv.csv", quote=FALSE, na="",
row.names=FALSE)
#
# (d) write.csv2 usa (;) come separatore di campo e (,) come separatore dei decimali
write.csv2(mytable, file="C:/Rdati/write_csv2.csv", quote=FALSE, na="",
row.names=FALSE)
#

```

Le prime sei righe servono semplicemente a generare la tabella impiegata come esempio.

Questo è il contenuto della tabella **mytable** [2] con il punto (.) come separatore dei decimali e con gli identificativi/nomi di riga (1, 2, eccetera) generati automaticamente da **R**:

```

> mytable # mostra la tabella nella quale R impiega il punto (.) come separatore dei decimali

```

	id	sex	age	weight_kg	height_m
1	MT	M	69	76	1.78
2	GF	F	56	63	NA
3	MC	F	53	71	1.60
4	SB	M	28	73	1.78
5	FE	F	61	54	1.54
6	AB	M	46	92	1.84
7	RF	F	31	81	1.56

Dopo avere mostrato il contenuto della tabella:

- in **(a)** la funzione **write.table()** viene impiegata una prima volta per esportare i dati nel file `write_table_p.csv` nel quale è previsto il punto (.) come separatore dei decimali;
- in **(b)** la funzione **write.table()** viene impiegata una seconda volta per esportare i dati nel file `write_table_v.csv` nel quale è prevista la virgola (,) come separatore dei decimali;
- in **(c)** la funzione **write.csv()** viene impiegata per esportare i dati nel file `write_csv.csv` impiegando i separatori secondo le convenzioni vigenti nel *mondo anglosassone*, cioè la virgola (,) come separatore di campo [4] e il punto (.) come separatore dei decimali;
- in **(d)** la funzione **write.csv2()** viene impiegata per esportare i dati nel file `write_csv2.csv` impiegando i separatori secondo le convenzioni vigenti in *Europa*, cioè il punto e virgola (;) come separatore di campo e la virgola (,) come separatore dei decimali.

Da notare che il separatore dei decimali nelle funzioni **write.csv()** e **write.csv2()** è fissato all'interno delle funzioni stesse e non può essere cambiato.

I file generati sono salvati nella cartella `C:\Rdati\` e anche in questo caso possono essere aperti con un editor di testo per verificarne il contenuto e possono essere importati in *Excel* o in *OpenOffice calc* o in *LibreOffice calc*.

Dopo avere illustrato il metodo canonico per esportare i dati da **R** [5], aggiungo che in realtà esiste anche un metodo naïf ma a dire il vero molto rapido e perfettamente funzionante.

Supponiamo che vi interessi esportare, esattamente come è stata visualizzata nella `Console` di **R** con **mytable**, la tabella riportata qui sopra. Selezionate il testo della tabella in questo modo

	id	sezzo	anni	peso_kg	altezza_m
1	MT	M	69	76	1.78
2	GF	F	56	63	NA
3	MC	F	53	71	1.60
4	SB	M	28	73	1.78
5	FE	F	61	54	1.54
6	AB	M	46	92	1.84
7	RF	F	31	81	1.56

quindi copiatelo e incollatelo, aggiungendo un `\n` al termine dell'ultima riga, in un editor di file di testo come ad esempio il Blocco note di Windows, infine salvatelo nella cartella Rdati con il nome `naif` - lasciate pure che l'editor aggiunga l'estensione di

default `.txt` e quindi il nome completo del file sarà `naif.txt`.

Sappiamo che un file `.txt` è identico a un file `.csv` a parte, appunto, l'estensione del nome del file. In questo caso abbiamo un file di testo che impiega come separatore di campo un tabulatore `<TAB>` e come separatore dei decimali il punto `(.)`.

Se ora copiate e incollate nella Console di R questa riga

```
mynaif <- read.table("c:/Rdati/naif.txt", header=TRUE, sep="", dec=".")
```

dove tra le virgolette in **`sep=""`** ho avuto cura di inserire (anche se non lo vedete) il carattere `<TAB>`, ritrovate in **`mynaif`** i vostri dati:

```
> mynaif <- read.table("c:/Rdati/naif.txt", header=TRUE, sep="      ", dec=".")
> mynaif
  id sesso anni peso_kg altezza_m
1 MT     M   69    76    1.78
2 GF     F   56    63     NA
3 MC     F   53    71    1.60
4 SB     M   28    73    1.78
5 FE     F   61    54    1.54
6 AB     M   46    92    1.84
7 RF     F   31    81    1.56
```

In buona sostanza avete esportato i dati semplicemente copiandoli direttamente dalla Console di R e salvandoli in un file di testo - dal quale li potete rileggere con **R** ma anche con qualsiasi altro applicativo - senza digitare nemmeno una riga di codice!

[1] Vedere i post:

- [Importazione dei dati da un file .csv](#)
- [Importazione dei dati da un file di solo testo](#)
- [Importazione dei dati da un file .xls o .xlsx](#)

[2] Parliamo di **array** o **vettore** nel caso di dati numerici monodimensionali, disposti su una sola riga,

8	6	11	7
---	---	----	---

di **matrice** nel caso di **dati numerici** disposti su più righe e più colonne

8	9	15	14

6	7	18	12
11	8	17	13
7	4	19	17

e di **tabella** nei casi in cui il contenuto, disposto su più righe e più colonne, è rappresentato oltre che da **dati numerici**, anche da **testo** e/o **operatori logici**

M	7	9	VERO
F	3	12	VERO
F	5	10	FALSO

[3] Per il significato di "Software libero", di "Software open source" e di "Software di dominio pubblico" vedere: *The Free Software Foundation. GNU Operating System. Categories of free and nonfree software.*

<https://www.gnu.org/philosophy/categories.en.html>

[4] L'impiego della virgola come separatore di campo (,) è alla base della modalità `comma separated value` che definisce la struttura dati dei relativi file e dal cui acronimo è derivato il nome `.csv` dell'estensione.

[5] Un esempio che illustra come impiegare la funzione **`write.table()`** per esportare in un file `.csv` uno dei numerosi set di dati forniti con **R** e con i suoi pacchetti aggiuntivi è riportato nel post [Il set di dati ais](#).
